Dynamic Mandatory Engine

Table of Contents

- Purpose
- Supported Entities
- Rule Summary
- Runtime Safety
- Technical Documentation
- Maintenance Guide
- Condition Value Examples
  - Text
  - Boolean
  - OptionSet
  - Multi-select OptionSet
  - Lookup by name
  - Lookup by ID
  - Lookup entity type
- Rule Examples
  - No rule matches, use default
  - Two rules match, mandatory fields are merged
  - Multiple conditions in one rule
  - Multi-select overlap
  - Lookup by GUID
  - Lookup by name
  - Owner is user instead of team

Purpose

The Dynamic Mandatory Engine sets required fields in Dynamics forms based on JSON configuration stored per Location / Business Unit.

The active runtime configuration is read from:

```
Table:  ambcust_location
Field:  mhwrmb_mandatoryconfigjson
```

The engine exists to keep mandatory-field business rules out of individual form scripts and make them configurable as data.

Supported Entities

The current runtime mapping supports:

Form entity	Location / Business Unit lookup
`contact`	`nev_businessunitid`
`account`	`nev_businessunit`
`wrmb_portfolio`	`ambcust_locationid`

The OnLoad handler must be registered on the forms where dynamic mandatory fields should be active:

```
WRM.dynamicMandatoryEngine.initializeDynamicMandatoryFields
```

Rule Summary

At runtime, the engine:

1. Resolves the current form entity.
2. Resolves the Location / Business Unit lookup.
3. Loads JSON from `mhwrmb_mandatoryconfigjson`.
4. Selects the matching entity block from `config.entities`.
5. Resets all potentially mandatory fields to optional.
6. Evaluates all rules.
7. Merges fields from all matching rules.
8. Uses `default` only when no rule contributes fields.
9. Registers OnChange handlers for condition fields.

Rules are additive. There is no first-match-wins behavior.

Runtime Safety

The engine must not block form usage. If configuration is missing, invalid, or does not apply to the current form, no mandatory rules are applied.

Technical Documentation

Purpose

The Dynamic Mandatory Engine sets Dynamics form fields to required or optional based on a JSON configuration stored on the related Location / Business Unit record.

The objective is to avoid hardcoded mandatory-field logic in individual form scripts. Business-specific required-field rules are maintained as JSON data instead of duplicated JavaScript.

Runtime Components

Component	Purpose
`dynamicMandatoryEngine.js`	Generated WebResource bundle used by Dynamics forms.
`dynamicMandatoryEngine.ts`	TypeScript source of the engine.

Component	Purpose
`MandatoryConfig.entity.ts`	Dataverse table and field constants for configuration loading.
`condition.evaluator.ts`	Generic condition evaluation logic.
`crm.core.ts`	Shared Xrm helpers and JSON contract types.
`mandatory-config.contract.yaml`	Machine-readable contract for review and validation.

The Webpack entry is:

```
dynamicMandatoryEngine:
./WebResources/src/features/dynamicMandatory/dynamicMandatoryEngine.ts
```

The generated global namespace is:

```
WRM.dynamicMandatoryEngine
```

Public Form Handlers

The bundle exports two Dynamics-compatible handlers:

```
WRM.dynamicMandatoryEngine.initializeDynamicMandatoryFields
WRM.dynamicMandatoryEngine.applyDynamicMandatoryRules
```

initializeDynamicMandatoryFields

Use this handler on form `OnLoad`.

It performs the full initialization:

1. Resolve the form context.
2. Resolve the Location / Business Unit lookup for the current entity.
3. Load the JSON configuration from Dataverse.
4. Parse the JSON.
5. Apply the matching mandatory rules.
6. Auto-register OnChange handlers for all fields used in rule conditions.

applyDynamicMandatoryRules

This handler reapplies the mandatory logic.

The engine registers it automatically on all condition fields. It may also be registered manually if a form requires explicit re-evaluation from another event.

Dataverse Configuration Source

Configuration is stored on:

```
Table:  ambcust_location
Field:  mhwrmb_mandatoryconfigjson
```

The engine retrieves the field with:

```
Xrm.WebApi.retrieveRecord("ambcust_location", locationId, "?
$select=mhwrmb_mandatoryconfigjson")
```

The field value must contain a valid JSON document matching the mandatory configuration contract.

Location / Business Unit Resolution

The engine determines the Location / Business Unit lookup from the current form entity:

Current entity	Lookup field used to find `ambcust_location`
`contact`	`nev_businessunitid`
`account`	`nev_businessunit`
`wrmb_portfolio`	`ambcust_locationid`

If no lookup value exists, no rules are applied.

New entities must be added explicitly in `getBusinessUnitAttributeForForm`.

Runtime Rule Algorithm

The runtime starts with the form `OnLoad` handler:

```
WRM.dynamicMandatoryEngine.initializeDynamicMandatoryFields
```

The exact sequence is:

1. The handler receives the Dynamics `executionContext`.
2. The engine gets the current `formContext` from `executionContext.getFormContext()`.
3. The engine determines the current form entity with:

```
formContext.data.entity.getEntityName()
```

Example result:

```
contact
```

4. The engine resolves which form lookup contains the Location / Business Unit reference:

Form entity	Lookup read by the engine
`contact`	`nev_businessunitid`
`account`	`nev_businessunit`
`wrmb_portfolio`	`ambcust_locationid`

5. The engine reads the lookup value from the form.
6. If the lookup is empty, the engine stops and applies no required-field changes.
7. If the lookup has a value, the GUID is sanitized and used as the Location / Business Unit ID.
8. The engine checks the browser-session cache with key `location:{id}`.
9. If the config is already cached, the cached config is reused.
10. If the config is not cached, the engine loads the record from Dataverse:

```
Table: ambcust_location
ID:    Location / Business Unit lookup ID
Field: mhwrmb_mandatoryconfigjson
```

11. The JSON text from `mhwrmb_mandatoryconfigjson` is parsed.
12. If the JSON is empty, invalid, or does not contain `entities`, the parsed config is `null` and no rules are applied.
13. If the JSON is valid, the engine reads the matching entity block:

```
const entityLogicalName = formContext.data.entity.getEntityName();
const entityConfig = config.entities[entityLogicalName];
```

Example:

```
const entityConfig = config.entities["contact"];
```

This maps to the JSON block:

```
{
  "entities": {
    "contact": {
      "default": ["lastname"],
      "rules": []
    }
```

```
    }
}
```

14. If there is no block for the current entity, the engine stops and applies no changes.
15. The engine builds a set of all fields that could be mandatory:
    - all fields from `entityConfig.default`,
    - all fields from every `rules[].mandatory`.
16. The engine resets all these fields to optional with `required = false`.
17. The engine evaluates every rule in `entityConfig.rules`.
18. A rule matches only when all conditions in its `condition` array match.
19. If a rule matches, all fields from that rule's `mandatory` array are added to a merged list.
20. Duplicate field names are ignored in the merged list.
21. After all rules are evaluated:
    - if the merged list contains fields, those fields are set to required,
    - if the merged list is empty, the fields from `entityConfig.default` are set to required.
22. After applying the rules, the engine registers OnChange handlers for all fields used in rule conditions.
23. When one of those condition fields changes, the engine reruns only the apply part:

```
WRM.dynamicMandatoryEngine.applyDynamicMandatoryRules
```

This reloads the configuration through the same cache-aware loading logic and reapplies the reset/evaluate/merge/default sequence.

Important behavior:

- Rules are additive.
- There is no first-match-wins behavior.
- `default` is a fallback only.
- `default` fields are not automatically added when one or more rules match.
- Before every apply run, all potentially required fields from `default` and all rules are reset to optional first.

Rule Evaluation

Conditions inside one rule are combined with AND.

Example:

```
{
  "name": "active_swiss_company",
  "mandatory": ["emailaddress1", "websiteurl"],
  "condition": [
    { "field": "statecode", "operator": "eq", "value": 0 },
    { "field": "wrm_country", "operator": "eq", "value": "CH" }
  ]
}
```

The rule matches only if both conditions match.

If `condition` is missing or empty, the rule always matches.

Supported Operators

Operator	Meaning
`eq`	Equal.
`ne`	Not equal.
`in`	Actual value is in the configured value list. For multi-select fields, any overlap is enough.
`isnull`	Empty or not set.
`isnotnull`	Set and not empty.
`notnull`	Alias for `isnotnull`.

Unknown operators do not match.

Supported Field Types

Field type	Expected JSON value
Text	String, compared case-insensitively.
Boolean	`true` / `false`; string representations are also tolerated.
OptionSet	Numeric option value.
Multi-select OptionSet	Array of numbers or strings.
Lookup	GUID, lookup name, or object with `id`, `name`, `entityType`.

GUIDs are normalized by removing braces and lowercasing.

Lookup Projections

Lookup conditions support dot notation:

```
{ "field": "primarycontactid.id", "operator": "eq", "value": "a1b2c3d4-1111-2222-3333-444455556666" }
```

Supported projections:

- `.id`
- `.name`
- `.entityType`

Without projection, lookup comparison behaves as follows:

- If the configured value is a GUID, compare by lookup ID.
- Otherwise compare by lookup name.

OnChange Wiring

During initialization, the engine reads all condition fields from the entity configuration and registers an OnChange handler on each base attribute.

Example:

```
{ "field": "primarycontactid.name", "operator": "eq", "value": "Jane Doe" }
```

The engine registers OnChange on:

```
primarycontactid
```

When one of these fields changes, the mandatory rules are evaluated again.

Cache Behavior

The loaded configuration is cached in the browser session by Location / Business Unit ID.

Cache key examples:

```
location:aaaaaaaa-bbbb-cccc-dddd-eeeeeeeeeeee
location:null
```

Implications:

- Reopening or refreshing the form loads the current configuration.
- Changes to the JSON may not affect an already opened form immediately if the same Location / Business Unit was already loaded in that browser session.
- If a tester changes JSON configuration, refresh the browser tab before retesting.

Error Handling

The engine is defensive and must not block form usage.

No rules are applied when:

- no Location / Business Unit lookup is available,
- the Location / Business Unit lookup has no ID,
- the Dataverse record cannot be loaded,
- the JSON field is empty,
- the JSON is invalid,
- no entity block exists for the current entity,
- a configured field is not available on the current form.

Missing attributes are ignored.

Required Dynamics Form Setup

For every form using this feature:

1. Add the generated `dynamicMandatoryEngine.js` WebResource as a form library.
2. Register this OnLoad handler:

```
WRM.dynamicMandatoryEngine.initializeDynamicMandatoryFields
```

3. Enable `Pass execution context as first parameter`.
4. Ensure all fields referenced by `mandatory` and `condition` are available on the form.
5. Ensure the form contains the Location / Business Unit lookup needed for the entity.

Deployment Notes

Build with:

```
npm run build
```

The generated bundle is:

```
WebResources/dist/dynamicMandatoryEngine.js
```

Deploy it as the corresponding Dynamics JavaScript WebResource according to the project packaging process.

Maintenance Guide

Purpose

This guide explains how to maintain the JSON configuration for the Dynamic Mandatory Engine.

Use this document when a business user or administrator asks to change which fields are required for a specific Location / Business Unit.

Where The Configuration Is Stored

The active configuration is stored in Dynamics:

```
Table:  ambcust_location
Field:  mhwrmb_mandatoryconfigjson
```

Each Location / Business Unit can have its own JSON configuration.

Local example files are available for review and development:

```
WebResources/src/config/BusinessUnitMandatoryConfig.basic.example.json
WebResources/src/config/BusinessUnitMandatoryConfig.lookup-guid.example.json
MandatoryConfigJson-QuickGuide.md
```

The local files are examples only. The runtime engine reads the JSON from Dynamics.

Supported Entities

The current engine supports these entity blocks:

Entity block in JSON	Dynamics source
`contact`	Contact form.
`account`	Company / account form.
`wrmb_portfolio`	Portfolio form.

If a new entity must be supported, a developer must add the entity-to-Location lookup mapping in the engine.

JSON Root Structure

The JSON must have this root structure:

```
{
  "version": 1,
  "entities": {
    "contact": {
      "default": ["lastname"],
      "rules": []
    }
  }
}
```

Required root properties:

Property	Meaning
`version`	Must currently be `1`.
`entities`	Object containing one block per Dynamics entity logical name.

Entity Block

Each entity block can contain:

Property	Meaning
`default`	Fields required when no rule contributes mandatory fields.
`rules`	Business rules that conditionally require fields.

Example:

```
{
  "default": ["lastname", "emailaddress1"],
  "rules": [
    {
      "name": "external_contact",
      "mandatory": ["parentcustomerid", "mobilephone"],
      "condition": [
        { "field": "wrm_contacttype", "operator": "eq", "value": "external" }
      ]
    }
  ]
}
```

Rule Structure

Each rule has:

Property	Required	Meaning
`name`	Yes	Unique technical rule name within the entity block.
`mandatory`	Yes	Fields that become required when the rule matches.
`condition`	No	AND-combined conditions. Empty or missing means the rule always matches.

How Rules Are Applied

The engine evaluates all rules for the current entity.

Important behavior:

- All matching rules are merged.
- If two rules match, the result is the union of both `mandatory` lists.
- `default` is used only when no matching rule contributes mandatory fields.
- The engine first clears required flags from all fields listed in `default` and all rules, then applies the current result.

Example:

```
{
  "default": ["name"],
  "rules": [
    {
      "name": "vip",
      "mandatory": ["ownerid"],
      "condition": [{ "field": "wrm_isvip", "operator": "eq", "value": true }]
    },
    {
      "name": "missing_country",
      "mandatory": ["wrm_country"],
      "condition": [{ "field": "wrm_country", "operator": "isnull" }]
    }
  ]
}
```

If both rules match, `ownerid` and `wrm_country` are required. `name` from `default` is not automatically added.

If the default fields must always remain required, repeat them in every rule's `mandatory` list.

Operators

Use only these operators:

Operator	Usage
`eq`	Value equals configured value.
`ne`	Value does not equal configured value.

Operator	Usage
`in`	Value is in configured list.
`isnull`	Field is empty.
`isnotnull`	Field is not empty.
`notnull`	Same as `isnotnull`.

Value Examples

This section contains copy/paste examples for common configuration cases.

Text

```
{ "field": "wrm_country", "operator": "eq", "value": "CH" }
```

Text is compared case-insensitively.

Boolean

```
{ "field": "wrm_isvip", "operator": "eq", "value": true }
```

OptionSet

```
{ "field": "customertypecode", "operator": "eq", "value": 1 }
```

Use the numeric option value, not the label.

Multi-select OptionSet

```
{ "field": "wrm_tags", "operator": "in", "value": [101, 202] }
```

For multi-select fields, `in` matches when at least one selected value overlaps with the configured list.

Lookup By Name

```
{ "field": "primarycontactid", "operator": "eq", "value": "Jane Doe" }
```

This is readable but less stable than comparing by ID.

Lookup By ID

```
{ "field": "primarycontactid.id", "operator": "eq", "value": "a1b2c3d4-1111-2222-3333-444455556666" }
```

This is the recommended approach for stable lookup comparisons.

Lookup Entity Type

```
{ "field": "ownerid.entityType", "operator": "eq", "value": "systemuser" }
```

Useful for owner fields where the value can be a user or a team.

Rule Examples

No Rule Matches, Use Default

If no rule matches, the fields in `default` become required.

```
{
  "default": ["name"],
  "rules": [
    {
      "name": "vip_requires_primary_contact",
      "mandatory": ["primarycontactid"],
      "condition": [
        { "field": "wrm_isvip", "operator": "eq", "value": true }
      ]
    }
  ]
}
```

If `wrm_isvip` is not `true`, the only required field is:

```
["name"]
```

Two Rules Match, Mandatory Fields Are Merged

If multiple rules match, the engine uses the union of all matching `mandatory` lists.

```
{
  "default": ["name"],
  "rules": [
```

```
    {
      "name": "prospect_requires_primary_contact",
      "mandatory": ["primarycontactid"],
      "condition": [
        { "field": "customertypecode", "operator": "eq", "value": 1 }
      ]
    },
    {
      "name": "vip_requires_owner",
      "mandatory": ["ownerid"],
      "condition": [
        { "field": "wrm_isvip", "operator": "eq", "value": true }
      ]
    }
  ]
}
```

If both conditions match, the required fields are:

```
["primarycontactid", "ownerid"]
```

`name` from `default` is not added, because `default` is fallback only.

Multiple Conditions In One Rule

Conditions inside one rule are AND-combined.

```
{
  "name": "active_prospect_requires_address",
  "mandatory": ["primarycontactid", "address1_line1"],
  "condition": [
    { "field": "customertypecode", "operator": "eq", "value": 1 },
    { "field": "statecode", "operator": "eq", "value": 0 }
  ]
}
```

This rule matches only when `customertypecode` is `1` and `statecode` is `0`.

Multi-select Overlap

Use `in` when at least one selected value should match.

```
{
  "name": "tagged_customer_requires_owner",
  "mandatory": ["ownerid"],
  "condition": [
    { "field": "wrm_tags", "operator": "in", "value": [101, 202] }
```

```
    ]
}
```

If the form value contains either `101` or `202`, the rule matches.

Lookup By GUID

Prefer GUID checks for stable lookup rules.

```
{
  "name": "specific_primary_contact_requires_phone",
  "mandatory": ["telephone1"],
  "condition": [
    {
      "field": "primarycontactid.id",
      "operator": "eq",
      "value": "a1b2c3d4-1111-2222-3333-444455556666"
    }
  ]
}
```

Lookup By Name

Lookup name checks are readable, but they can break if the record name changes.

```
{
  "name": "specific_primary_contact_name_requires_phone",
  "mandatory": ["telephone1"],
  "condition": [
    { "field": "primarycontactid", "operator": "eq", "value": "Jane Doe" }
  ]
}
```

Owner Is User Instead Of Team

Use `.entityType` when the same lookup can point to different table types.

```
{
  "name": "user_owned_record_requires_owner_note",
  "mandatory": ["wrm_ownernote"],
  "condition": [
    { "field": "ownerid.entityType", "operator": "eq", "value": "systemuser" }
  ]
}
```

Maintenance Workflow

1. Identify the Location / Business Unit whose mandatory logic must change.
2. Open the related `ambcust_location` record in Dynamics.
3. Edit field `mhwrmb_mandatoryconfigjson`.
4. Copy the current JSON into an editor.
5. Validate that the JSON is syntactically valid.
6. Change only the relevant entity block and rule.
7. Verify all field names are Dynamics logical names.
8. Save the JSON back to `mhwrmb_mandatoryconfigjson`.
9. Refresh the Dynamics form in the browser.
10. Test all affected rule paths.

Validation Checklist

Before saving a JSON change:

- Is the JSON valid?
- Is `version` set to `1`?
- Are entity names logical names such as `contact`, `account`, `wrmb_portfolio`?
- Are all field names logical names?
- Are all mandatory fields present on the target form?
- Are OptionSet values numeric codes, not labels?
- Are lookup comparisons using `.id` where stability matters?
- Are rule names unique within the entity block?
- Does each rule contain a clear `mandatory` list?
- Is `default` understood as fallback only?

Troubleshooting

No fields become required

Check:

- The form has a Location / Business Unit lookup value.
- The related `ambcust_location` record contains JSON in `mhwrmb_mandatoryconfigjson`.
- The JSON is valid.
- The JSON contains an entity block matching the current form entity logical name.
- The configured fields are available on the form.
- The browser tab was refreshed after changing JSON.

The wrong fields are required

Check:

- Multiple rules may match and are merged.
- `default` is only used when no rule contributes fields.
- Condition fields use the correct logical names.
- OptionSet values use numeric codes.

- Lookup conditions compare the intended value: ID, name, or entity type.

Changes do not appear immediately

The engine caches configuration per Location / Business Unit in the browser session.

Refresh the form browser tab after changing `mhwrmb_mandatoryconfigjson`.

A field is configured but not required

Check:

- The field exists on the form.
- The field logical name is correct.
- The field is not disabled by another form script or business rule.
- The rule condition actually matches.

Best Practices

- Keep rules small and specific.
- Prefer lookup `.id` comparisons for stable behavior.
- Use clear rule names that explain the business case.
- Do not use display names or labels as field names.
- Avoid relying on `default` if fields must always be mandatory; repeat those fields in matching rules.
- Test every changed rule with at least one positive and one negative case.